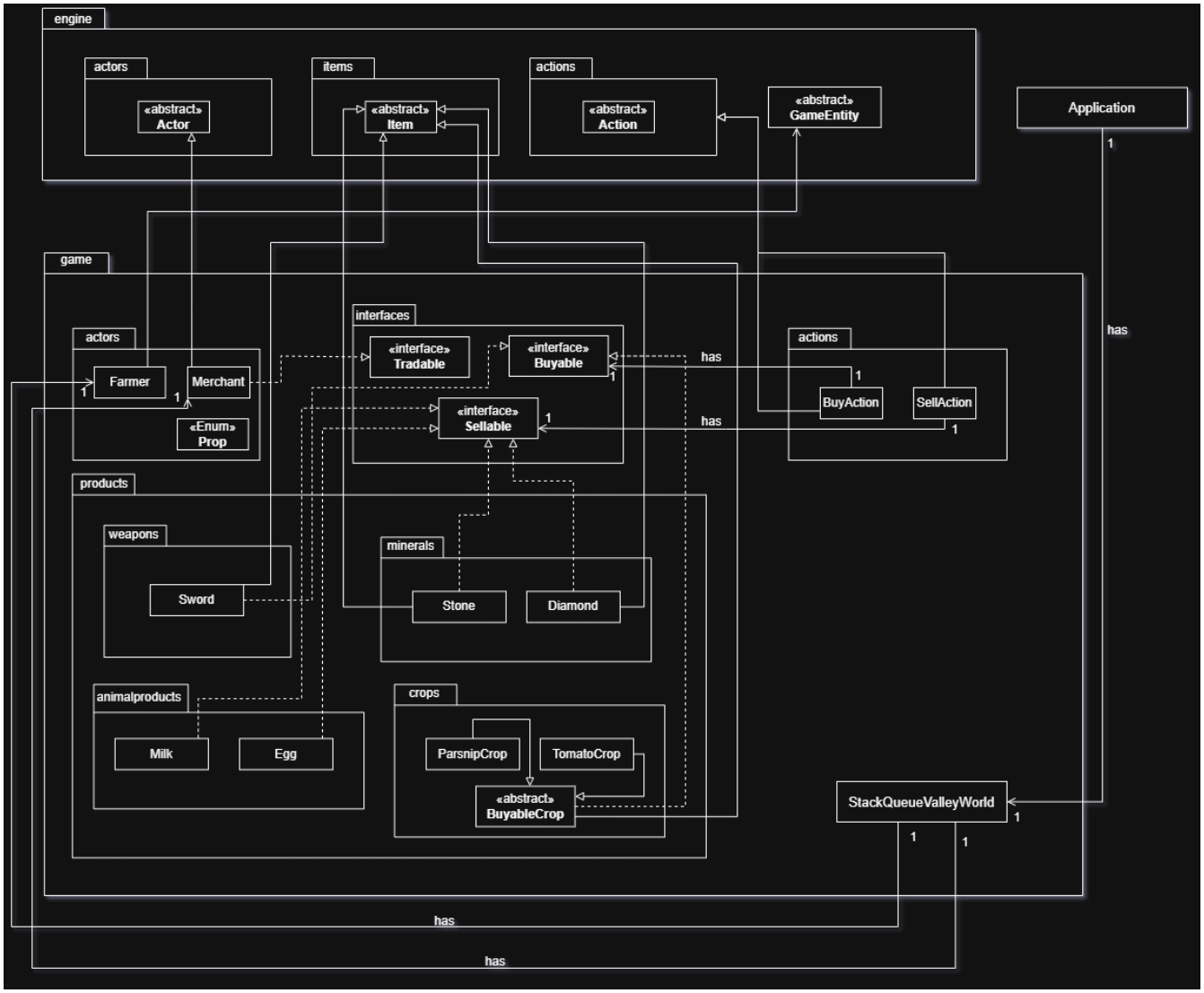
REQ3 The Merchant

Design Diagram

REQ3 UML Class Diagram



A The Merchant & Trading

Design 1 Not Selected:

Create a concrete class for the merchant, without extending from any super class. Creating brand new wandering and other actions can be taken by merchant, and build-up new private instance 'coin'. Instead of handling the trade, the buy actions will be handled by the farmer side.

Advantages	Trade-offs
- Simple & straightforward, each class contain their own implementations without much relations with another class may reduce coupling. - No inheritance required, reduce the code complexity and avoid potential hidden bugs when modify super class.	- Increase code redundancy, Merchant class share same properties with Actor super class, only contain one new instance. - Increase code duplication, large amount of repeated code will be required if same class created for similar implementations. - Reduce extensibility, any later updates with 'actor-like' role will need to manually adjust to the merchant class. - Decrease maintainability and testability, as each class takes own attributes and behaviors, great amount of base class for testing will be created.

Design 2 Selected:

Create a concrete class that extends from Actor abstract super class, override the allowed action and play turn methods with own implementation. Implement the Wanderable interface for the merchant actor to perform wandering behaviors, and create a new Enum called Prop with value COIN to store the number of coins with BasicStatistic class. Create a Tradable interface, and inject the trade method for both selling and buying between the merchant and farmer together through method overloading.

Advantages	Trade-offs
- Improve code reusability and, as it utilizes the given abstract Actor class, which provided with basic attributes and behaviors. More types of actors can easily create and reuse from super class without generate own class. - Promote code extensibility, new actors can be added by extending Actor without changing existing ones. New behaviours can be added as new interfaces. - New type of merchant also allows to alter their wander behaviors without much effort through method overriding from the Wanderable interface. - Wanderable and Tradable are small and focused. Classes implement only the capabilities they need. - Interfaces (Wanderable, Tradable) allow high-level code to depend on abstractions rather than concrete classes.	- If Tradable forces both buy and sell on every implementer, a class that can only sell (or only buy) must implement an unused method. - Increase coupling, sub-class now inherit from super abstract class, any change in super abstract class will affect the implementation in sub-class. - Merchant still handles turn decisions, wandering and trading. playTurn can become a large method that decides between all of them.

B The sellable items

Design 1 Not Selected:

Create an abstract class Sellable which not inherit from Item abstract class to handle an Item object that is sellable. Construct a soldBy method to perform the sell action from the farmer to the merchant. Let Egg, Milk, Stone and Diamond classes extend from this Sellable class.

Advantages	Trade-offs
- Simple & Straightforward, the Sellable class should only be extended by the other objects that can be sold. - Obey to the responsibility organization, as the Sellable class will only focus on implementing the selling actions or any effects happens after the selling actions.	- Code duplication, since Sellable shares common properties with the Item class, create new Sellable class result in duplicated entities and methods. - Risk of multi-level of inheritance, when the sellable items own additional characteristic such as the GameEntity class, as Java does not support multiple inheritance, the extensibility of Sellable class reduce. - Missing core functions, many items are in farmer's inventory, so it is not a good choice perform adding actions of items in farmer class, since this may result in many conditional statements checking the type of objects. There is no method return the actions of the sellable items, reduce the reusability when it needs to be performed as an item.

Design 2 Not Selected:

Create an abstract class Sellable which inherit from Item abstract class to handle an Item object that is sellable. Construct a soldBy method to perform the sell action from the farmer to the merchant. Let Egg, Milk, Stone and Diamond classes extend from this Sellable class.

Advantages	Trade-offs
- Sellable separates sellable items from non-sellable ones (e.g. quest items), so non-sellable items don't carry an unused soldBy method. - Shared selling logic (check ownership, transfer coins, remove item) is written once in Sellable and inherited by all four items. - Adding new items or changing the shared sale logic in one place is simple.	- soldBy mixes the item's own data with a transaction (coin transfer, inventory removal). The item is now responsible for how a sale happens, not just for being sellable. - Because Sellable is an abstract class, an item can't be sellable and also inherit another class (single inheritance). A Sellable interface would be more flexible. - Adding a new category (e.g. Edible items like Parsnip and Tomato) is hard with single inheritance, and the hierarchy may deepen or need restructuring. - Risk to violate the responsibility organization, the sellable items should only handle the effects of selling, instead of holding the trade effects between the merchant and the farmer.

Design 3 Selected:

Create a Sellable interface, containing the method of soldBy only handling the effect of selling an item from the farmer to the merchant, without any additional manipulation the Actor objects. The sellable items Egg, Milk,

Stone and Diamond class extends from the given Item super class and implements the Sellable interface, override the soldBy method.

Advantages	Trade-offs
- Align with Single Responsibility Principle, soldBy only handles the effect of the sale on the item, and leaves coin transfer and inventory changes to the actors or a trade method. Each item class is responsible only for its own sale effect. - A new sellable item (e.g. Wool) extends Item, implements Sellable and overrides soldBy, with no change to existing classes. - Sellable has a single method, so implementers aren't forced to depend on anything unused. Non-sellable items simply don't implement it. - Callers (e.g. a trade method) depend on the Sellable abstraction, not on Egg, Milk, Stone or Diamond. Not manipulating Actor objects also keeps items decoupled from concrete actors. - Shared actor-side logic (coin transfer, inventory removal) lives elsewhere, e.g. in Tradable, so it isn't duplicated across items. - Using an interface instead of an abstract class means items can combine other capabilities (e.g. Sellable and Consumable) while still extending Item. It is a good fit for composition.	- Using an interface instead of an abstract class means items can combine other capabilities (e.g. Sellable and Consumable) while still extending Item. It is a good fit for composition. - If soldBy takes concrete Farmer or Merchant parameters, the abstraction leaks. Taking no actor parameters, or only Actor, avoids this. - If the sale effect is identical across all four items, the same soldBy body is repeated four times. Interfaces can't hold shared state, though a default method can fix this.

C The buyable items

Design 1 Not Selected:

Create an abstract Buyable class without inherit from the super class Item. With its own specific method boughtBy, let Parsnip, Tomato and Sword class extends from the Buyable abstract class and override the boughtBy method.

Advantages	Trade-offs
- Simple & Straightforward, the Buyable class should only be extended by the other objects that can be sold. - Clear responsibility management, as the Buyable class will only takes all entities and behaviors of an object that can be bought.	- Reduce code reusability, as Item and Buyable class will focus on serving the objects that can be bought, some responsibilities between them may overlap. - Letting the Parsnip and Tomato class extends from this will result in multi-level inheritance, and since these two classes already take responsibility to the consumption effect, so these classes can be swollen and take multiple responsibility, neglect Single Responsibility Principle. - Lots of method rewrite or potentially reject features of polymorphism. Since Buyable class do not have any relationship with Item class, any methods or instances in Item class cannot be overridden or used by the Buyable object, neglect the Liskov Substitution Principle.

Design 2 Selected:

Create a Buyable interface, which maintain the method boughtBy to perform the effects of buy actions. Then, create an abstract BuyableCrop class in crops package and implement the Buyable interface, as the main operations and effects between the Parsnip and Tomato crop are similar. Create ParsnipCrop and TomatoCrop class extends from the BuyableCrop, to take the responsibility to perform the effects after the buy actions. Reuse the GameEntity inherited by Item super class to represent the price of the crops.

Advantages	Trade-offs
- Buyable defines only the buy-effect contract. BuyableCrop holds the shared crop behaviour, and ParsnipCrop and TomatoCrop handle only what is specific to each crop after purchase. - A new buyable crop (e.g. CarrotCrop) extends BuyableCrop and only supplies its differences, with no change to existing classes. - ParsnipCrop and TomatoCrop can be used anywhere a BuyableCrop or Buyable is expected, as long as they keep the base class's contract for boughtBy. - The similar operations between Parsnip and Tomato live once in BuyableCrop, so they aren't copied into both subclasses. - Buyable can be reused for any purchasable thing (tools, animals, seeds), not just crops. BuyableCrop is reusable by every crop type.	- If boughtBy takes concrete parameters (e.g. Farmer), the abstraction leaks. Using Actor (or no actor at all) keeps it decoupled. - If the shared logic in BuyableCrop has to change for one crop, it risks affecting all subclasses. Hard-coded crop-specific checks in the base class would violate OCP.

Design 3 Not Selected:

Create a Buyable interface contains the boughtBy method. Create new ParsnipCrop and TomatoCrop class to implement this interface, override the boughtBy method to perform their own boughtBy effects.

Advantages	Trade-offs
- A new buyable crop or other item (e.g. CarrotCrop, a tool) implements Buyable without changing any existing class.	- Code duplication, if the Parsnip and Tomato effects and fields (price, name) are largely similar, the same code is written twice, and every new crop copies it again. A fix in one place must be repeated in the others. - Behavior can't be reused between crops. Each class reimplements logic that a shared base (or a default method) could have provided once. - Each crop class still combines crop behavior with the buy effect. If the effects grow complex, the classes take on more than one reason to change. - Null method. If boughtBy takes parameters only some crops need, it would start forcing unused arguments on others.

D The BuyAction & SellAction

Design 1 Not Selected:

Create concrete classes for BuyAction and SellAction which both not inherit from any super class, constructing own execution and description method. Both classes will handle one Item object with the corresponding trader, which is the merchant.

Advantages	Trade-offs
- They can still use Buyable and Sellable for the item, so the item side is abstract. - Each class does one job: BuyAction performs a purchase and SellAction performs a sale. Each has its own execute and description logic, so a change to buying never touches selling.	- Code redundancy. Because both action classes shared major responsibilities as well as methods (action execution and description). Constructing own methods will increase number of duplicated codes. - Also, methods with same responsibility may share same name, without method overriding, it is risky to violate the connascence of naming. - There is also no shared type to substitute. The code that runs actions cannot treat BuyAction and SellAction uniformly, which is the whole purpose of Liskov Substitution Principle. - Reduce reusability, the shared parts (coin transfer, description format) can't be reused by other actions, and the actions can't be reused by systems that expect a common action type. - Since Item object do not have effect of selling or buying, so the additional effect will be taken place inside the action, potentially result in breach of Single Responsibility Principle.

Design 2 Selected:

Create concrete classes for BuyAction and SellAction which both inherit from the Action abstract super class in engine package, overriding the execute and menu description method. Both Action class hold Buyable and Sellable item respectively to adhere to the tradeBy method overloading in Merchant class.

Advantages	Trade-offs
- Improve reusability, as method override from super class to sub classes, avoid unnecessary risk of connascence of naming. - Align with Liskov Substitution Principle, as each buy and sell action has their own effects, the Buyable and Sellable interface perform as an abstract contract between the higher level and lower level module. Avoid the uses of instanceof to differentiate the types of the object. - High-level engine code depends on the Action abstraction, and the actions depend on Buyable and Sellable abstractions rather than on ParsnipCrop, Egg and so on. - Easy to add new buyable and sellable items (they just implement the interface) and new actions (they extend	- Each new trade direction or item category means another overload in Merchant, and Merchant grows as the trade rules do. Any item that is both buyable and sellable creates an ambiguous overload call unless the static type is chosen carefully. - Adding a new item category (neither Buyable nor Sellable) requires a new action type and a new tradeBy overload in Merchant, so the merchant is modified for every new kind.

Advantages	Trade-offs
Action). The menu and engine pick them up automatically.	

Conclusion

Overall, the merchant will be the new actor manage the trades with the farmer. Which items in merchant's inventory will performing as the buyable items from the aspect of farmer, the famer need sufficient coins to purchase the items, maybe with extra effects. Items in farmer's inventory will perform as sellable items from the aspect of the farmer, farmer can choose to sell to the merchant to receive coins, extra effects may be executed. The items in merchant's inventory is infinite, as long as the farmer has enough coins, he can buy unlimited stuff.

For 4 implementations, design 2, 3, 2 and 2 are chosen respectively. For implementation A, design 2 is selected because it obeys to Liskov Substitution Principle where inheritance from the Actor super class enable the merchant to perform the behaviors as an Actor without any exceptions since it's an upcasting. Implementing the interfaces instead of multiple levels of inheritance enable the classes to reduce coupling and increase extensibility (Open-Close Principle). As long as the method in interface remains the same, the high- and low-level module won't be mutually affected (Dependency Inversion Principle). As the merchant only deals with trades, method overloading maintains this responsibility (Single Responsibility Principle).

For implementation B and C, design 3 and 2 is similar, but with different goals. For sellable item, since they do not share similar operational logic, interface is enough to construct an abstract contract between the modules (Dependency Inversion Principle). But for buyable items, especially for crop, they share almost the same logic, which implements interface only will violate the DRY principle, so create new BuyableCrop abstract class is appropriate to hold the common logic, and construct sub-routines to receive the minor difference between the crops.

For implementation D, design 2 is selected because buy and sell action should not directly depends on the Item, which may increase the dependency and coupling between classes. Instead, they should depend on the Buyable and Sellable interface, so any changes in action classes or item classes won't affect each other (Dependency Inversion Principle). Additionally, the inheritance from Action abstract super class can reduce code redundancy (DRY), and promote polymorphism – since buy and sell action can be used as Action class object through safe upcasting (Liskov Substitution Principle).

All in all, although the selected designs obey to most of the design principles, but the trade-offs should not be ignored, further improvement and code refactoring is required.